WEEK 2: SUPERMARKET MANAGEMENT SYSTEM

1. Scenario

The Supermarket Management System is a database application used to maintain information about products and their categories. Each product belongs to a specific category, and the system manages product details such as product name, price, and stock quantity.

The system helps the supermarket organize product information efficiently and makes it easier to search, update, and generate category-wise product reports.

---

2. Category Table

The **CATEGORY** table stores information about the different categories available in the supermarket.

Each category is identified by a unique CATEGORY_ID.

```
SQL> CREATE TABLE CATEGORY (CATEGORY_ID VARCHAR2(10) PRIMARY KEY, CATEGORY_NAME VARCHAR2(50) NOT NULL UNIQUE);

Table created.

SQL> DESC CATEGORY;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CATEGORY_ID                               NOT NULL VARCHAR2(10)
 CATEGORY_NAME                             NOT NULL VARCHAR2(50)
```

3. Product Table

The **PRODUCT** table stores information about the products available in the supermarket.

Each product has a unique PRODUCT_ID and is connected to a category using CATEGORY_ID.

```
SQL> CREATE TABLE PRODUCT (PRODUCT_ID VARCHAR2(10) PRIMARY KEY, PRODUCT_NAME VARCHAR2(100) NOT NULL, CATEGORY_ID VARCHAR2(10) NOT NULL, PRICE NUMBER(10,2) NOT NU
_QUANTITY NUMBER(10) NOT NULL, CONSTRAINT FK_PRODUCT_CATEGORY FOREIGN KEY (CATEGORY_ID) REFERENCES CATEGORY(CATEGORY_ID), CONSTRAINT CHK_PRODUCT_PRICE CHECK (PRI
CONSTRAINT CHK_PRODUCT_STOCK CHECK (STOCK_QUANTITY >= 0));

Table created.

SQL> DESC PRODUCT;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PRODUCT_ID                                NOT NULL VARCHAR2(10)
 PRODUCT_NAME                              NOT NULL VARCHAR2(100)
 CATEGORY_ID                               NOT NULL VARCHAR2(10)
 PRICE                                     NOT NULL NUMBER(10,2)
 STOCK_QUANTITY                            NOT NULL NUMBER(10)
```

4. Define Relationships and Constraints

Relationship

A single category can contain multiple products.

Therefore, the relationship between **CATEGORY** and **PRODUCT** is:

**One-to-Many (1 →)*

CATEGORY

|

| 1

|

|------< PRODUCT

*

This follows the same relationship structure as your example PDF.

Primary Key

- CATEGORY_ID uniquely identifies every category in the CATEGORY table.
- PRODUCT_ID uniquely identifies every product in the PRODUCT table.

Foreign Key

CATEGORY_ID in the PRODUCT table is used as a **Foreign Key**.

It connects each product with its corresponding category.

NOT NULL Constraint

Important fields such as:

- CATEGORY_NAME
- PRODUCT_NAME
- CATEGORY_ID
- PRICE
- STOCK_QUANTITY

should not be left empty.

CHECK Constraint

The price of a product must be greater than zero.

PRICE > 0

Stock quantity cannot be negative.

STOCK_QUANTITY >= 0

This prevents invalid product information from being entered.

UNIQUE Constraint

Each category must have a different category name.

Duplicate category names are not allowed.

Constraint Verification

The USER_CONSTRAINTS view is used to verify the constraints defined on the CATEGORY and PRODUCT tables. The query displays the constraint name, constraint type, and status. All the constraints are shown as ENABLED.

```
SQL> SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, STATUS
  2  FROM USER_CONSTRAINTS
  3  WHERE TABLE_NAME IN ('CATEGORY', 'PRODUCT');

CONSTRAINT_NAME                C STATUS
------------------------------ - --------
SYS_C007170                    C ENABLED
SYS_C007171                    P ENABLED
SYS_C007172                    U ENABLED
SYS_C007173                    C ENABLED
SYS_C007174                    C ENABLED
SYS_C007175                    C ENABLED
SYS_C007176                    C ENABLED
CHK_PRODUCT_PRICE              C ENABLED
CHK_PRODUCT_STOCK              C ENABLED
SYS_C007179                    P ENABLED
FK_PRODUCT_CATEGORY            R ENABLED

11 rows selected.
```

Inserting Values

Values are inserted into both the CATEGORY and PRODUCT tables to create sample supermarket records.

After inserting the records, the contents of both tables are displayed to verify that the data has been added successfully.

```
SQL> INSERT INTO CATEGORY VALUES ('CAT001', 'Grocery');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY VALUES ('CAT002', 'Beverages');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY VALUES ('CAT003', 'Snacks');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY VALUES ('CAT004', 'Personal Care');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY VALUES ('CAT005', 'Household');

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM CATEGORY;

CATEGORY_I CATEGORY_NAME
---------- --------------------------------------------------
CAT001     Grocery
CAT002     Beverages
CAT003     Snacks
CAT004     Personal Care
CAT005     Household
```

Product Table:

The PRODUCT table contains sample supermarket products along with their category, price, and available stock quantity. The inserted records are displayed to verify that the

data has been added successfully.

```
SQL> SELECT * FROM PRODUCT;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
P001
Rice 5kg
CAT001            350             25

P002
Wheat Flour 1kg
CAT001             65             30

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------

P003
Sugar 1kg
CAT001             55             20

P004
Orange Juice

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
CAT002            120             15

P005
Green Tea
CAT002            180             12

P006
```

```
PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
Mineral Water
CAT002             25             50

P007
Potato Chips
CAT003             40             35


PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
P008
Biscuits
CAT003             30             40

P009
Chocolate
CAT003             80             25

PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------

P010
Shampoo
CAT004            220             18

P011
Bath Soap
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
CAT004             45             30

P012
Toothpaste
CAT004            110             20

P013

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
Dishwash Liquid
CAT005            150             16

P014
Floor Cleaner
CAT005            190             14


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
P015
Laundry Detergent
CAT005            280             10


15 rows selected.
```

CRUD Operations

Insertion Record

A new product can be added to the PRODUCT table using the INSERT command.

```
SQL> INSERT INTO PRODUCT
  2  VALUES ('P016', 'Coffee Powder', 'CAT002', 250, 20);

1 row created.
```

Update Price

The UPDATE command is used to modify the price of an existing product.

```
SQL> UPDATE PRODUCT
  2  SET PRICE = 270
  3  WHERE PRODUCT_ID = 'P016';

1 row updated.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM PRODUCT
  2  WHERE PRODUCT_ID = 'P016';

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
P016
Coffee Powder
CAT002            270             20
```

Delete a Product

The DELETE command is used to remove a particular product record from the PRODUCT table. The Coffee Powder product is deleted from the table and the deletion is verified using a SELECT query.

```
SQL> DELETE FROM PRODUCT
  2  WHERE PRODUCT_ID = 'P016';

1 row deleted.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM PRODUCT
  2  WHERE PRODUCT_ID = 'P016';

no rows selected

SQL>
```

Category-Wise Product Report

The JOIN operation is used to combine the CATEGORY and PRODUCT tables using CATEGORY_ID. The report displays each product along with its corresponding category, price, and stock quantity.

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      P.PRODUCT_ID,
  5      P.PRODUCT_NAME,
  6      P.PRICE,
  7      P.STOCK_QUANTITY
  8  FROM PRODUCT P
  9  JOIN CATEGORY C
 10  ON P.CATEGORY_ID = C.CATEGORY_ID;

CATEGORY_I CATEGORY_NAME                                      PRODUCT_ID
---------- -------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
CAT001     Grocery                                            P001
Rice 5kg
       350             25

CAT001     Grocery                                            P002
Wheat Flour 1kg
        65             30

CATEGORY_I CATEGORY_NAME                                      PRODUCT_ID
---------- -------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------

CAT001     Grocery                                            P003
Sugar 1kg
        55             20

CAT002     Beverages                                          P004
Orange Juice
```

```
CATEGORY_I CATEGORY_NAME                                           PRODUCT_ID
---------- ---------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
       120             15

CAT002     Beverages                                               P005
Green Tea
       180             12

CAT002     Beverages                                               P006

CATEGORY_I CATEGORY_NAME                                           PRODUCT_ID
---------- ---------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
Mineral Water
        25             50

CAT003     Snacks                                                  P007
Potato Chips
        40             35


CATEGORY_I CATEGORY_NAME                                           PRODUCT_ID
---------- ---------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
CAT003     Snacks                                                  P008
Biscuits
        30             40

CAT003     Snacks                                                  P009
Chocolate
        80             25
```

```
CATEGORY_I CATEGORY_NAME                                      PRODUCT_ID
---------- -------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
        45             30

CAT004     Personal Care                                      P012
Toothpaste
       110             20

CAT005     Household                                          P013

CATEGORY_I CATEGORY_NAME                                      PRODUCT_ID
---------- -------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
Dishwash Liquid
       150             16

CAT005     Household                                          P014
Floor Cleaner
       190             14


CATEGORY_I CATEGORY_NAME                                      PRODUCT_ID
---------- -------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE STOCK_QUANTITY
---------- --------------
CAT005     Household                                          P015
Laundry Detergent
       280             10
```

Counting Number of Products in Each Category

The COUNT function is used to determine the total number of products available in each supermarket category. The GROUP BY clause groups the products according to their category.

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      COUNT(P.PRODUCT_ID) AS TOTAL_PRODUCTS
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7  ON C.CATEGORY_ID = P.CATEGORY_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME;

CATEGORY_I CATEGORY_NAME                                TOTAL_PRODUCTS
---------- -------------------------------------------------- --------------
CAT005     Household                                                 3
CAT004     Personal Care                                             3
CAT001     Grocery                                                   3
CAT002     Beverages                                                 3
CAT003     Snacks                                                    3

SQL>
```

Finding Highest-Priced Product in Each Category

The MAX function is used to identify the highest price of a product within each supermarket category. The GROUP BY clause groups the products category-wise before finding the maximum price.

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      MAX(P.PRICE) AS HIGHEST_PRICE
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7  ON C.CATEGORY_ID = P.CATEGORY_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME;

CATEGORY_I CATEGORY_NAME                                      HIGHEST_PRICE
---------- -------------------------------------------------- -------------
CAT005     Household                                                    280
CAT004     Personal Care                                                220
CAT001     Grocery                                                      350
CAT002     Beverages                                                    180
CAT003     Snacks                                                        80

SQL>
```

Display Categories Having More Than 2 Products

The GROUP BY and HAVING clauses are used to identify categories containing more than two products. The COUNT function counts the products in each category, and the HAVING clause filters the categories based on the specified condition.

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      COUNT(P.PRODUCT_ID) AS TOTAL_PRODUCTS
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7  ON C.CATEGORY_ID = P.CATEGORY_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
  9  HAVING COUNT(P.PRODUCT_ID) > 2;

CATEGORY_I CATEGORY_NAME                                    TOTAL_PRODUCTS
---------- -------------------------------------------------- -------------
CAT005     Household                                                      3
CAT004     Personal Care                                                  3
CAT001     Grocery                                                        3
CAT002     Beverages                                                      3
CAT003     Snacks                                                         3
```

Average Product Price Category-Wise

The AVG function is used to calculate the average price of products available in each supermarket category. The GROUP BY clause groups the products category-wise and

calculates the average price for each category.

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      ROUND(AVG(P.PRICE), 2) AS AVERAGE_PRICE
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7  ON C.CATEGORY_ID = P.CATEGORY_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME;

CATEGORY_I CATEGORY_NAME                                      AVERAGE_PRICE
---------- -------------------------------------------------- -------------
CAT005     Household                                                 206.67
CAT004     Personal Care                                                125
CAT001     Grocery                                                   156.67
CAT002     Beverages                                                 108.33
CAT003     Snacks                                                        50

SQL>
```

Conclusion

The Supermarket Management System efficiently manages product and category information using an Oracle database. The project demonstrates table creation, primary and foreign key relationships, constraints, data insertion, CRUD operations, and category-wise reports. SQL functions such as JOIN, COUNT, MAX, AVG, GROUP BY, and HAVING are used to retrieve and analyze product information. The system helps maintain organized product records and supports efficient data management.